

CoScript Project Report

INDEX

1. Abstract
 2. Problem Statement
 3. Proposed System (Overview & Modules)
 4. System Requirement (Software & Hardware)
 5. Software Used
 6. System Analysis (Entity-Relationship Diagram)
 7. System Design (UML Diagrams Overview)
 8. Frontend Screens
 9. Backend Screens
 10. Code Sample
 11. Testing
 12. Implementation
 13. Maintenance
 14. Conclusion
 15. Bibliography
-

1. ABSTRACT

CoScript is a real-time collaborative code editor designed to provide an interactive and synchronized coding environment for developers. Built with a modern web stack—React.js for the frontend and Node.js with Socket.io for the backend—the platform ensures seamless, low-latency collaboration across multiple users in shared coding rooms.

The platform provides essential modules such as user authentication (Google OAuth & Email/Password), secure room creation, dynamic file exploration, a multi-language execution environment (supporting JavaScript, Python, Java, C, C++, and TypeScript), and an integrated terminal. By centralizing the collaborative process and offering real-time code execution, CoScript boosts productivity and streamlines remote pairing and technical interviews.

2. PROBLEM STATEMENT

In the modern software development landscape, developers frequently need to collaborate remotely on codebases, pair program, or conduct technical interviews. However, using isolated IDEs combined with screen sharing tools leads to a disjointed experience where only one person can effectively write code at a time. The lack of real-time synchronization causes workflow interruptions and decreases productivity.

Key Challenges:

- **Fragmented Collaboration:** Remote developers struggle to write and test code simultaneously without conflict.
- **Environment Setup Overhead:** Setting up identical development environments across different machines for pair programming is time-consuming.
- **Lack of Integrated Execution:** Many collaborative editors lack in-browser code execution for multiple languages, forcing users to switch contexts to test their logic.

- **Data Synchronization:** Maintaining state and resolving conflicts across multiple active users requires robust real-time communication protocols.

CoScript solves these problems by providing a unified, cloud-based editor where developers can instantly join a session, write code together concurrently, and execute it within the same browser interface.

3. PROPOSED SYSTEM

3.1 Overview & Goals

CoScript is a web-based real-time collaborative editor using WebSocket technology to ensure instantaneous updates across all connected clients.

- **Real-Time Collaboration:** Goal is to provide a zero-latency editing experience where multiple users can type simultaneously in the same file.
- **In-Browser Execution:** Goal is to securely compile and run code in multiple languages directly within the editor workspace.
- **Secure Room Management:** Objective is to allow hosts to create secure session rooms, manage join requests, and restrict access.
- **Integrated Terminal & File Explorer:** Goal is to simulate a full IDE environment in the browser.

3.2 Modules

- **Authentication Module:** Secure Sign Up / Login using JWT and Google OAuth integration.
- **Room Management Module:** Host-controlled rooms with join request approvals, user kicking capabilities, and real-time active user tracking.
- **Editor Module:** Powered by Monaco Editor (the core of VS Code) providing syntax highlighting, error checking, and real-time cursor tracking.
- **Execution Module:** Backend service capable of spawning processes to compile and run C, C++, Java, Python, and JavaScript code.
- **Terminal Module:** An interactive node-pty based terminal integrated into the frontend workspace for command-line access.

4. SYSTEM REQUIREMENTS

4.1 Software Requirements

Frontend:

- React.js (Vite/Create React App)
- Tailwind CSS (UI styling)
- Monaco Editor (@monaco-editor/react)
- Socket.io-client
- React Router

Backend:

- Node.js & Express.js
- Socket.io (WebSocket handling)
- Mongoose (MongoDB ODM)
- Node-pty (Terminal emulation)
- JSON Web Tokens (JWT) & bcrypt

Database:

- MongoDB (Storage for users and room metadata)

4.2 Hardware Requirements

Development Scope:

- Processor: Intel i5/i7 or AMD Ryzen equivalent.
- RAM: Minimum 8GB.
- Storage: 100GB SSD.
- OS: Windows, macOS, or Linux.

5. SOFTWARE USED

- **React.js:** Primary framework for building the Single Page Application (SPA), ensuring smooth client-side routing and fast UI updates via Virtual DOM.
- **Tailwind CSS:** Utility-first CSS framework used for designing scalable and responsive UI components (Activity Sidebar, Context Menus, File Explorer).
- **Socket.io:** Facilitates the bi-directional event-based communication between the Node.js server and React client, critical for real-time code changes.
- **Express.js:** Web framework used to build RESTful API endpoints for authentication and file system management.
- **MongoDB:** NoSQL database used for persistent data storage (Users, Rooms).

6. SYSTEM ANALYSIS (ERD Overview)

Entities and Attributes:

1. **User:** id (PK), name, email, password, authProvider, createdAt.
2. **Room:** roomId (PK), hostId (FK), hostSocketId, code, createdAt.

Relationships:

- A **User** can create (host) multiple **Rooms**.
- A **Room** contains a list of approvedUsers and pendingRequests referencing the **User** constraints.

7. SYSTEM DESIGN (UML Overview)

The architecture follows a Client-Server model.

- **Use Case:** A user logs in, creates a room, and shares the RoomID. Another user requests to join. The host approves the request. Both users edit the code in real-time, and either user can click "Run" to test the output.
- **Class Structure:** Separation of React Components (EditorPage.jsx, WorkspaceExplorer.jsx, VSCodeTerminal.jsx) communicating via Socket.js contexts to backend services (TerminalService.js, CodeRunner.js, Room.js).

(Note: Depending on your university guidelines, you can insert actual UML diagrams drawn via draw.io or lucidchart into this section).

8. FRONTEND SCREENS

1. **Login/Registration Screen:** Features Google OAuth and traditional email/password inputs with a modern UI.
2. **Dashboard/Home Screen:** Where the user can either input a Room ID to join or click "Create Room" to start a new session.
3. **Editor Workspace:**
 - Left Panel: File Explorer & Activity Sidebar.
 - Main Panel: Monaco Code Editor.
 - Bottom Panel: Integrated Terminal / Output window.
 - Right Panel: Active Users / Chat / Settings.
4. **Modals:** Host approval modals for joining users, Settings modal for theme/font adjustments.

9. BACKEND SCREENS

Because the backend is a Node.js API, there is no direct graphical interface. The backend is monitored via server terminal logs (e.g., "Server running on port 3001", "User connected", "Room joined") and MongoDB GUI clients (like MongoDB Compass) for database monitoring.

10. CODE SAMPLE

Server.js Execution Endpoint (Node.js):

```
app.post("/execute", async (req, res) => {
  const { language, code } = req.body;
  if (!language || !code) return res.status(400).json({ error: "Missing required fields" });

  try {
    const result = await executeCode(language, code); // Compiles and runs code dynamically
    res.json(result);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

WebSocket Connection (React Client):

```
socket.on("code_update", ({ code }) => {
  setCodeContent(code);
});

const handleCodeChange = (newCode) => {
  setCodeContent(newCode);
  socket.emit("code_change", { roomId, code: newCode });
};
```

11. TESTING

- **Unit Testing:** Ensuring individual utility functions (like executeCode parsing and language mapping) return expected results.
- **Integration Testing:** Testing the WebSocket events (e.g., ensuring join_request successfully triggers a host notification).

- **System Testing:** End-to-end testing where two browser windows simulate two users collaborating in a room.
- **Black-Box Testing:** Ensuring invalid Language selections or malformed code payloads are handled gracefully by the server without crashing.

12. IMPLEMENTATION

The system is built to run locally or be deployed to cloud services.

1. Requires Node v14+ and MongoDB.
2. The user installs dependencies via npm install for both client and server.
3. Using the package script npm run dev, Concurrently boots up the Express server on port 3001 and the React Vite client on port 3000.
4. Developers can securely code together from multiple browser tabs or disparate networks if hosted (e.g., Render/Vercel).

13. MAINTENANCE

- **Scalability Updates:** As active rooms grow, the architecture can be updated from basic Socket.io in-memory tracking to Redis adapters for a distributed WebSocket network.
- **Security Updates:** Routine patches for npm dependencies, adjusting JWT expirations, and ensuring the CodeRunner.js module runs user code securely (e.g., Docker sandboxing) to prevent malicious arbitrary command execution on the host machine.

14. CONCLUSION

CoScript successfully demonstrates the capacity to merge rich text editing, low-latency web sockets, and backend remote execution into a single, cohesive developer platform. By mimicking the structure of VS Code within a web browser, it allows users to overcome the limitations of isolated IDEs. The robust room authentication flow and multi-language support provide a highly flexible environment for technical interviews and pair programming, solidifying its place as a comprehensive remote collaboration tool.

15. BIBLIOGRAPHY

- React Documentation: <https://react.dev/>
- Socket.io Documentation: <https://socket.io/docs/v4/>
- Node.js Documentation: <https://nodejs.org/en/docs/>
- Monaco Editor Repository: <https://microsoft.github.io/monaco-editor/>
- MongoDB Documentation: <https://www.mongodb.com/docs/>